Listen

Learning Doc 11: Stage 2, the Voice Agent
One section per issue, written as each issue closes

Implementation, stage 2 of 6

started 4 November 2026

Abstract

This document grows during stage 2: when an issue closes, its section is added here while the lessons are fresh, and the stage's last issue (S2-19) polishes the whole. Each section is taught from zero: the problem first, then the idea, an analogy and where it stops being true, the real code and real output, the usual mistakes, and a quick check (answers at the end). So far it covers the six issues finished first: M1, M2, M3, M4, M6 and S2-08.

Contents

1 Before you start **3**

2 M1: pinning the tools that judge our code **3**
2.1 The problem . . . 3
2.2 The idea: pin . . . 3

3 M2: strict YAML, refusing duplicate keys **4**
3.1 The problem . . . 4
3.2 The idea: change one method of the parser . . . 4
3.3 Moved, not copied . . . 4
3.4 How the tests are written . . . 4

4 M3: an exit code with one meaning **5**
4.1 The problem . . . 5
4.2 The fix: 78 . . . 5
4.3 Why 78: the sysexits.h convention . . . 5
4.4 The guard test . . . 5

5 M4: a race test that can fail **6**
5.1 The problem . . . 6
5.2 Finding the real race first . . . 6
5.3 Making a rare race happen every time . . . 6
5.4 Proving the test works: break the code on purpose . . . 6

6 M6: a README that matches its folder **7**

7 S2-08: uv and the lock file **7**
7.1 The problem . . . 7
7.2 How the lock file is made . . . 7
7.3 What writes it, and what only reads it . . . 7

8 Key takeaways so far **8**

9 Answers **8**

1 Before you start

Doc 10
Everything from stage 1: the package and its command line, settings, logging, events, the bus, workers, the supervisor, locks, fakes and pytest.

An issue and a PR
An issue describes one piece of work (its file in `docs/issues/`); a pull request is the change that does it. `Closes #12` in a PR closes issue 12 when the PR is merged.

A dependency
A library our code needs. `pyproject.toml` lists the ones we ask for; they may need further libraries of their own.

A version number
`0.17.3` is MAJOR.MINOR.PATCH: patches fix bugs, minor versions add features, major versions may break things.

2 M1: pinning the tools that judge our code

2.1 The problem

`pyproject.toml` said `dev = ["pytest>=8.0", "ruff>=0.6"]`: "this version *or anything newer*". Every install, including each fresh CI machine, fetched the newest release that day. A new ruff release can add a check; CI then turns red on a commit that only touched the README, with no code change at all.

2.2 The idea: pin

Pinning fixes one exact version:

```
dev = ["pytest==9.2.0", "ruff==0.17.3"]
```

Written	pip / uv may install
`ruff>=0.6`	anything from 0.6 up, so the newest
`ruff==0.17.3`	exactly 0.17.3 (pinned)
`ruff~=0.17.3`	0.17.3, 0.17.4 … but not 0.18 ("compatible")

Only the tools that *judge* the code were pinned: if they change, CI's verdict changes. Libraries the code merely uses are pinned later by the lock file (Section 7).

Updated before starting. The issue file was written with 0.17.1 and 9.1.1. On the day M1 started, 0.17.3 and 9.2.0 were out, and unpinned CI was already using them, which is exactly the problem. The rule "update the issue file before starting" caught it.

Analogy

Agreeing with a teacher which *edition* of the textbook the exam uses. *Where it breaks:* editions change rarely; tools release every few weeks, so a pin must be raised on purpose from time to time (Dependabot now proposes those upgrades as pull requests).

Quick check 1 (answer in Section 9)

With `ruff==0.17.3` pinned and ruff 0.18.0 released tomorrow, which version does the next CI run install, and what must you do to use 0.18.0?

3 M2: strict YAML, refusing duplicate keys

3.1 The problem

PyYAML's `safe_load` silently keeps the *last* copy of a repeated key. Writing `thresholds:` twice in `settings.yaml` threw away the first block, and its values quietly returned to defaults. The new test showed the bug before any fix existed:

```
E       Failed: DID NOT RAISE SettingsError
```

3.2 The idea: change one method of the parser

A YAML *loader* builds Python dicts from the file. A small subclass overrides the one method that builds a mapping, and refuses a key it has already seen:

```
class _StrictLoader(yaml.SafeLoader):
    def construct_mapping(self, node, deep=False):
        seen = set()
        for key_node, _ in node.value:
            key = self.construct_object(key_node, deep=deep)
            if key in seen:
                raise _DuplicateKey(key, key_node.start_mark.line + 1)  # lines from 1
            seen.add(key)
        return super().construct_mapping(node, deep=deep)
```

Real result: `Can't start: settings.yaml:4: duplicate key 'thresholds'`, exit code 78 (after M3). The line is the *second* occurrence: the one to delete.

3.3 Moved, not copied

`settings.py` lost its own reading and parsing code (8 lines removed) and now calls the shared `load_yaml` (3 lines). Reading a **diff**: lines with `-` were removed, lines with `+` added, unmarked lines are context. The old messages for unreadable files and broken YAML stayed word for word, which is why all 71 old tests passed without edits.

Why move it now rather than later? Fixing it inside `settings.py` would have been the smaller change; a well-known rule says build for reuse only once a second user exists. Here the second user, S2-01's command files, was planned and two steps away, and the check is subtle code that should not exist in two copies. Both choices were defensible; the lesson is to say the trade-off in the PR, which is now a review rule in `CODING_STANDARDS.md`.

3.4 How the tests are written

A test file `test_*.py` holds functions `test_*`; each calls the code and `asserts` the result. `with pytest.raises(Error, match="..."):` passes only if that error, with that message, is raised inside the block. A fixture such as `tmp_path` is requested by naming it as a parameter; each test gets a fresh one. `@pytest.mark.parametrize` runs one test once per row of a table.

Watch out

Inside `pytest.raises(ValueError)` an expected `ValueError` makes the test *pass*: it is the outcome you asked for. Without `match`, *any* `ValueError` passes, even one raised for a different reason.

Quick check 2 (answer in Section 9)

After M2, someone writes `mode:` twice in `settings.yaml`. What appears in the terminal, and with which exit code does `listen run` end?

4 M3: an exit code with one meaning

4.1 The problem

When a program ends it hands back one number. The launcher (S2-17) will only see that number. Code **2** meant two things: argparse uses it for a wrong command (`listen bogus`), and we used it for a bad `settings.yaml`. The launcher could not tell "you typed a bad command" from "fix your settings".

4.2 The fix: 78

```
EXIT_BAD_SETTINGS = 78  # EX_CONFIG: the owner must fix settings.yaml; restarting would
    not help
```

Everything else already used the *name*, so one line changed. Real runs: `listen bogus` exits 2, a bad `settings.yaml` exits 78.

4.3 Why 78: the sysexits.h convention

`sysexits.h` is a list of exit codes from BSD Unix that many command-line programs follow. It starts at 64 to stay clear of the small numbers programs use freely.

Code	Name	Meaning
64	`EX_USAGE`	the command was used wrongly
65	`EX_DATAERR`	the input data was wrong
66	`EX_NOINPUT`	an input file is missing or unreadable
69	`EX_UNAVAILABLE`	a needed service is unavailable
70	`EX_SOFTWARE`	an internal error (a bug)
71	`EX_OSERR`	an operating-system error
74	`EX_IOERR`	an input/output error
75	`EX_TEMPFAIL`	temporary failure: try again later
77	`EX_NOPERM`	permission denied
78	`EX_CONFIG`	configuration error

(Also 67 no such user, 68 no such host, 72 a system file missing, 73 cannot create a file, 76 protocol error.) Python offers these names as `os.EX_CONFIG` on Linux and Mac, but not on Windows: only `os.EX_OK` exists there, so `app.py` writes 78 itself.

Why it is done this way

75 is the one code where restarting makes sense; 78 is one where it never does. Choosing codes with a known meaning lets the launcher's rules follow from the number alone.

4.4 The guard test

```
def test_unknown_command_exits_2(capsys):
    with pytest.raises(SystemExit) as exit_info:
        main(["bogus"])
    assert exit_info.value.code == 2
```

```
    assert "invalid choice: 'bogus'" in capsys.readouterr().err
```

- `"bogus"` is a deliberately fake command. argparse ends the program by *raising* `SystemExit(2)`, so the test catches it.
- `exit_info.value` is the caught exception; `.code` its exit code.
- `capsys` records printed text; `.err` is the error stream (stderr), where argparse writes; `in` checks the phrase appears somewhere in it.

It passed from the start: it is a *guard* that pins argparse's 2, so the "bad settings must not be 2" test stays meaningful.

Quick check 3 (answer in Section 9)

Windows denies Listen the microphone, and that stops the agent. Which code fits, and why not 78?

5 M4: a race test that can fail

5.1 The problem

The stage 1 review found that the bus's concurrency test passed even with the lock removed. A test that cannot fail protects nothing.

5.2 Finding the real race first

An experiment before writing the test: 8 threads subscribe 8 inboxes to a *brand-new* event type at the same moment, 20 trials each way:

```
lock=True:  inboxes that got the event, per trial (of 8): [8]
lock=False: inboxes that got the event, per trial (of 8): [1]
```

Without the lock, each thread sees "no list for this type yet", makes its own new list, and each overwrites the last: 7 subscriptions vanish. This is a **check-then-act** race: between checking and acting, another thread changes the answer. The old test never created a new list, so it could not see it.

5.3 Making a rare race happen every time

Creating a list normally takes far too little time for threads to collide reliably. The test slows that one moment down on purpose and starts all threads together:

```
def slow_new_list():
    time.sleep(0.01)
    return []

bus._subscribers = defaultdict(slow_new_list)   # private, on purpose
start = threading.Barrier(8)                    # all 8 threads go at the same moment
```

5.4 Proving the test works: break the code on purpose

The lock was removed locally (never committed) and the test run: it failed with `assert [0, 0, 0, 0, 0, 0, ...] == [1, 1, 1, 1, 1, 1, ...]`. With the lock back: 20 of 20 runs passed. Breaking code on purpose to check that a test notices is called **mutation testing**; here it was done by hand.

Analogy

Testing a smoke alarm by holding a lit match under it rather than waiting for a real fire. *Where it breaks:* the match is not a real fire; the slowed-down factory is not the real timing. It proves the lock protects the window, not that the window is ever this wide in practice.

Quick check 4 (answer in Section 9)

Why does a test that touches a private attribute such as `_subscribers` need a comment, according to the project's coding standards?

6 M6: a README that matches its folder

A README is the first thing a reader opens in a folder; a stale one misleads. `agent/README` still called `pyproject.toml` "added when we build", and never mentioned `uv.lock`, `.python-version`, `settings.yaml` or `logs/`. The fix was one line per file. The habit it teaches: when a change adds a file to a folder, its README gets a line in the same PR.

7 S2-08: uv and the lock file

7.1 The problem

Stage 2 brings big libraries (faster-whisper, onnxruntime, numpy). With only `>=` ranges, a silent upgrade could change speed or behaviour between the laptop and CI.

7.2 How the lock file is made

`uv lock` reads the wishes in `pyproject.toml` (3 names), asks PyPI which versions exist, follows each library's own dependencies (pytest alone brought 5 more), and picks one version of each that satisfies every rule at once. That search is called **resolving**. The result, `uv.lock`, records 9 packages with exact versions, download URLs and a **hash** (a fingerprint of the exact file):

```
name = "pytest"
version = "9.2.0"
dependencies = [
    { name = "colorama", marker = "sys_platform == 'win32'" },
    { name = "iniconfig" }, { name = "packaging" }, { name = "pluggy" }, { name = "
    pygments" },
]
```

A universal lock. A test project with `uv add pandas` resolved 6 packages but installed 4: the lock lists numpy twice (one version for Python 3.11, another for 3.12 and later) and `tzdata` only for Windows. The lock covers every machine; an install takes the lines that match this one.

7.3 What writes it, and what only reads it

Command	With the lock
`uv lock`	resolves and writes it
`uv add` / `uv remove`	edits `pyproject.toml`, re-locks, installs
`uv sync`, `uv run`	re-lock first if it is missing or out of date
`uv sync --locked`	never writes it: fails if it is out of date (what CI runs)
`uvx tool`	runs a tool once without adding it to the project

`uv.lock` is committed (the recipe); `.venv` is not (the meal, rebuilt from the recipe). Real numbers: CI's agent job went from about 30–36 s to 22–28 s.

Watch out

The first CI run failed: `Unable to resolve action astral-sh/setup-uv@v10`. Many actions publish a moving major tag (`actions/checkout@v4`); setup-uv only publishes exact tags such as `v10.3.3`. Pinning the exact tag fixed it. Dependabot now proposes real, existing newer versions of actions, so nobody has to guess tags.

Quick check 5 (answer in Section 9)

Someone adds a library to `pyproject.toml` by hand and forgets `uv lock`. What does plain `uv sync` do in CI, what does `uv sync --locked` do, and why is the second safer?

8 Key takeaways so far

Issue	Mental model
M1	tools that judge the code get exact versions, raised on purpose
M2	refuse silently lost data; one shared loader, not two copies
M3	an exit code is a contract: one number, one meaning
M4	a test you never saw fail proves nothing; break the code to check it
M6	a folder's README changes in the same PR as the folder
S2-08	`pyproject.toml` says what is allowed; `uv.lock` says what was chosen

9 Answers

1. Still 0.17.3: the pin allows nothing else. To use 0.18.0, change the pin, run `uv lock` and `uv sync --extra dev`, run ruff and pytest, fix anything new, and merge that as its own PR (or merge the PR Dependabot opens for it).

2. `load_yaml` raises `YamlFileError` (`settings.yaml:N: duplicate key 'mode'`); `load_settings` turns it into `SettingsError`; `run_agent` prints `Can't start: settings.yaml:N: duplicate key 'mode'` and returns 78.

3. 77, `EX_NOPERM` (permission denied). 78 would send you to fix `settings.yaml`, which is not the problem.

4. Reaching into internals ties the test to details the public interface hides; the comment tells the next reader why the public interface was not enough, so they do not "tidy" it away or copy the pattern without reason.

5. Plain `uv sync` quietly re-locks with whatever is newest that day, so CI tests versions nobody reviewed. `--locked` refuses and fails, so the mistake is caught and the lock is updated deliberately in the PR.